Project Report – Virtrade

Members.

Name - GitHub Username

Austin Bushlack - a-bush6

Mihai-Alexandru Radu - MihaiAlexandru-Radu

Dylan Spektor - spektordylan

Harrison Kaufman - HarrisonKaufman

Ian O'Keefe - iokee4/ianokee (2 different accounts)

Project Description.

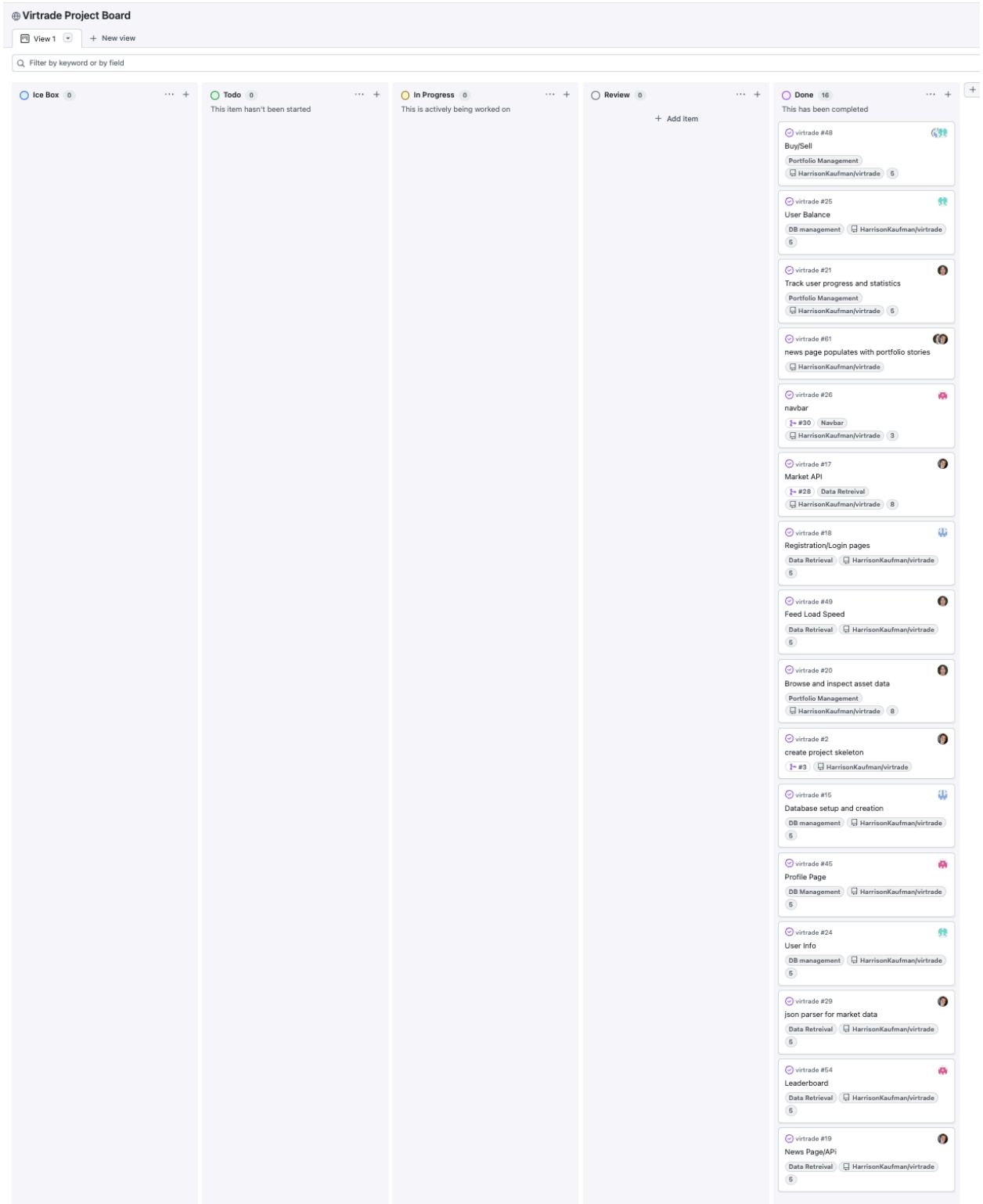
Virtrade is a paper-trading website, which allows you to see the effectiveness of trading techniques and practice them to compete with other users. On the website, you can bet on real stock data using fake money, avoiding the risk of financial loss.

The user can buy, and sell, and see the monetary information of stocks for a limited number of popular companies. First, there is a feed page that shows the lightweight version of the stocks, with the rises and falls of their value over a couple weeks. If you click on one of the graphs, you get a much more detailed version, with the ability to see the amount bought from the given company and to buy or sell them.

The home page has several news sources for each stock, each with a link directly to the source, to give the user more information before making buys or sells. From the navbar, the leaderboard is also available, where you can find the top or worst earning accounts currently active, based on the current order. The user also has the ability to see their current user information, including email, username, and current balance. There is also a "delete account" option.

Project Tracker – GitHub Project Board.

GitHub Project Board: https://github.com/users/HarrisonKaufman/projects/2



Video.

Demo Video: https://www.youtube.com/watch?v=TMyMcP47tnA

Version Control Software.

GitHub Repository: https://github.com/HarrisonKaufman/virtrade

Contributions.

Austin Bushlack: Made the initial draft for a lot of the POST functions when making an account and the Python user object (holding things like money), so that the database doesn't have to constantly update itself. Also worked on a lot of the buying and selling methods, making sure that they correspondingly update both the user object and the database. Most of the Python implementations use Flask API to properly connect to the Database, which was worked on to ensure that it could also be connected to the Stock API's (specifically Twelve Data and Finnhub API's).

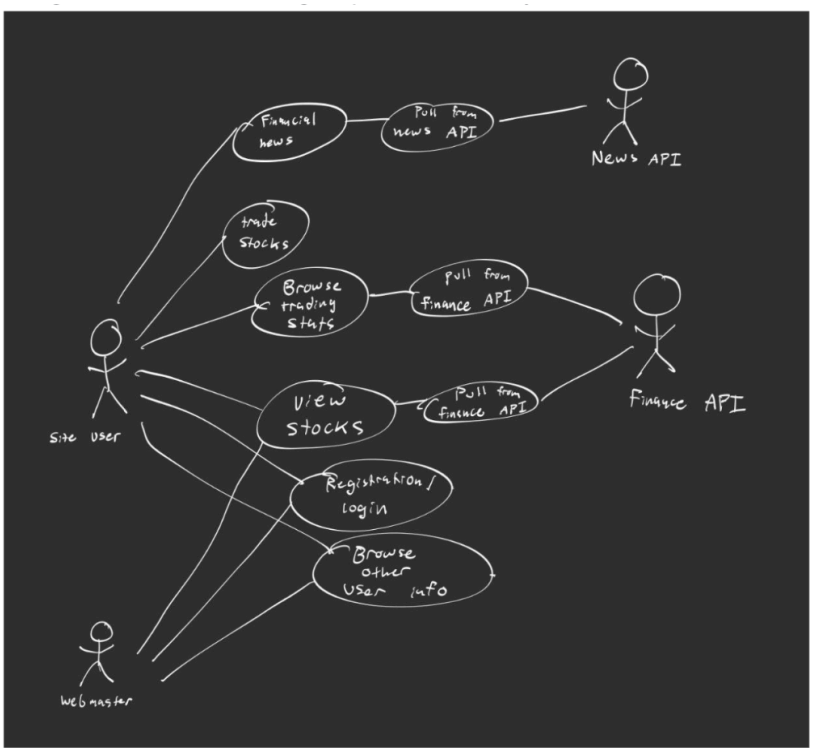
Mihai-Alexandru Radu: Worked on Navbar, with links to every page and account. Also made profile page partial, which shows an user's given profile details, with ability to delete account. Also made leaderboard, which orders based on deposit either ascending or descending based on the SQL database. Made fixes to the database to make sure that it works on Postgres and error handling for account registration. Refined initial UI to fit the dark mode setting that we went with, and made some of the pages more clean, with centered boxes for the account login and registration.

Dylan Spektor: Dylan worked both on the Lightweight graphs which show when you open up the Feed page and the complex ones when clicking on a specific company. (used the same two API's as Austin, and also on Partials for the graphs). He made sure in the JS index file that you can specify specific companies and the feed page will show corresponding graphs; worked on some caching so that the pages will load more quickly. Also worked on the homepage to improve UI by ensuring that user balance appears there and news pages are more organized.

Harrison Kaufman: Harrison worked on the majority of the API implementations. Made the initial commits for the Stock API and made a json parser so we can cleanly get its code for the future commits. Initialized the News API home page so that stock-related news appear on the first page, and also made it so that the user can see the given shares that they bought on the home page. Also made a majority of the Render work, initializing the cloud hosting and fixing bugs for it. Improved Navbar and worked on making cleaner pages in case of the API's failing.

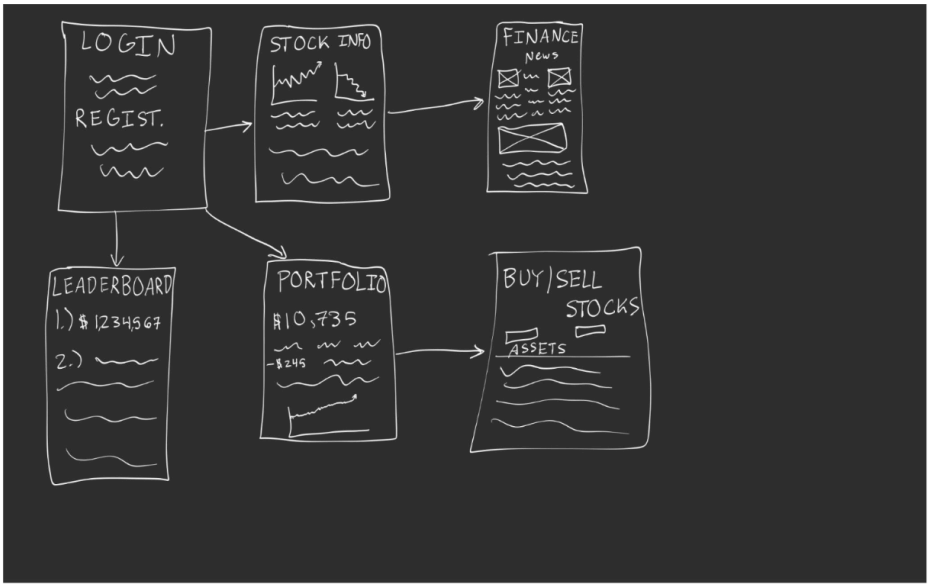
Ian O'Keefe: Ian made the original database schema and also worked on its implementation in the create SQL file, especially on the user and balance transactions datatables. Ian also worked on the Python Flask implementations for buy and sell, adding the POST methods for buying and selling in the Python counterpart also with sufficient error handling so that there are no errors present. He also worked on additional error handling with buying stocks, specifically in terms of checking stocks exist and that the user actually has enough money to buy a given amount (so that invalid buys/sells will not go through).

Use Case Diagram.

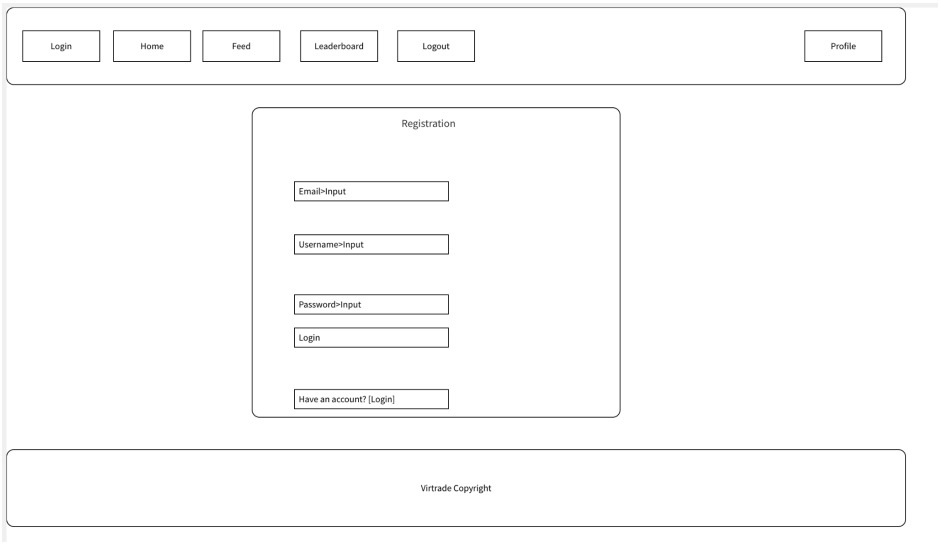


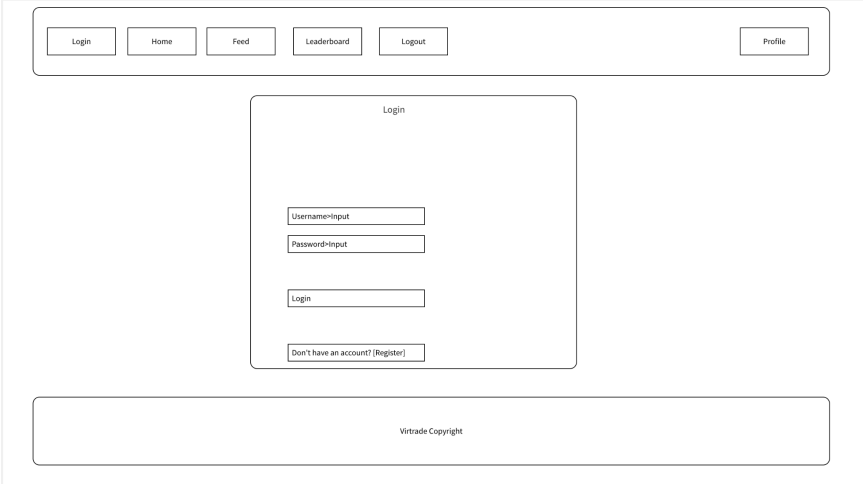
Wireframes.

First:

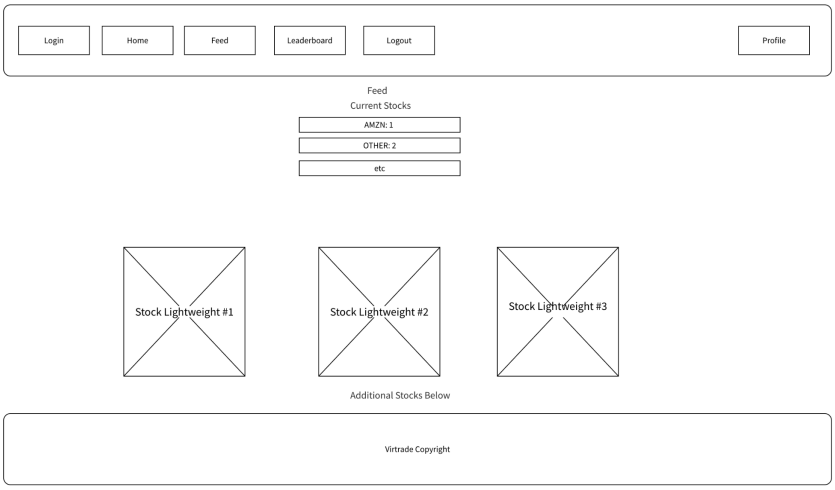

LOGIN
REGIST.
STOCK INFO
FINANCE
NEWS
LEADERBOARD
1.) $ 1234567
2.)
PORTFOLIO
$10,735
-$245
BUY/SELL
STOCKS
ASSETS

Final:


Login
Home
Feed
Leaderboard
Logout
Profile
Registration
Email>Input
Username>Input
Password>Input
Login
Have an account? [Login]
Virtrade Copyright


Login
Home
Feed
Leaderboard
Logout
Profile
Login
Username>Input
Password>Input
Login
Don't have an account? [Register]
Virtrade Copyright

Login
Home
Feed
Leaderboard
Logout
Profile
Home
[Username]
Balance: Balance
AMZN
See More
News Source #1 [Link]
News Source #2 [Link]
Other Stock
See More
News Source #1 [Link]
News Source #2 [Link]
Virtrade Copyright


Login
Home
Feed
Leaderboard
Logout
Profile
Feed
Current Stocks
AMZN: 1
OTHER: 2
etc
Stock Lightweight #1
Stock Lightweight #2
Stock Lightweight #3
Additional Stocks Below
Virtrade Copyright

Login Home Feed Leaderboard Logout Profile

Feed

Stock Name Number Owned: X

Heavy Graph [Detailed]

Buy X

Sell X

Virtrade Copyright

Login Home Feed Leaderboard Logout Profile

Leaderboard

v

1 [Name] Balance

2 [Name] Balance

3 [Name] Balance

4 [Name] Balance

5 [Name] Balance

Virtrade Copyright

Login Home Feed Leaderboard Logout Profile

Profile

Email: [Email]

Username: [Username]

Balance: [Balance]

Leaderboard Delete Account

Virtrade Copyright

Test Results.

A user has their correct information stored in the database for later use. A user cannot register without providing all required info (password, username etc.) The test data will be the information that we post to the database. The test environment will be a localhost The test results should be verification that the entered information is the same as that in the database The user acceptance testers will be our group working on the project Final test results will be input into report

1) Database/Register

Main User Feedback: Need to verify email is actually real, users could create accounts with same username (not direct feedback but realized through testing that it needed to be changed)

Our testing has shown that the database and the register seem to both be working correctly. Whenever, the user inputs an email, username, and password, the account is properly being created in the database, with all the valid credentials. One thing that we noticed that we decided to change later was that users could also create accounts with the same username (this has been changed later). This a very positive matter because the username is used for a lot of factors, most importantly determining the leaderboard. If multiple people have the same username, you wouldn't be able to tell which one is which. The users also wanted to test if they could break the logic somehow, which they've tried by inputting invalid emails (e.g. "abc") and still managed to create an account. That was the main primary feedback, specifically that users were able to just input random emails and still be able to make an account. Likewise, when we did testing, there was no valid email validation, which has been fixed now. The rest, such as the UI for the log-in and registration, faced no complaints and users were able to easily log-in with no database issues once they made an account. The primary changes that were requested and needed were the distinct usernames and valid emails.

2) Graphs

Main User Feedback: Rate limits can be finished rather quickly, especially when doing page reloads, need to create some form of data saving

The main problem that we did not foresee with users is that they could very easily load a page over and over again (which we found out rather quickly) and very quickly go through the rate limit. We were asked to improve this, by making some type of caching system which we have worked on to improve rate usage. However, the overall look of the graphs was not complained

about. The look was clean, and users could very clearly see when a stock went down or up in value with contrasting red and greens. Likewise, when checking that it actually matches the API info, there seems to have been no issue in this regard, with the API sending no errors and giving the same data. Overall, the only issue we had when doing the user testing was the rate limit, which we have tried to minimize. We also managed to almost completely reduce the loading times after some comments on having to wait multiple seconds to load the page.

3) UAT for a Stock Trade

Main User Feedback: Having to change pages constantly to see that trade went through is too much work. (Put number of stocks bought in given company on both pages instead)

Whenever the user tries to do a transaction, the purchase goes through succesfully, and does not fulfill the transaction if it costs more than what the user has in his balance. Likewise, when you go back to the default feed page, it shows what stocks you currently own and the number of the stocks you have. The main problem is that when you buy a certain amount of a given stock, you have to go back to the regular feed page to know that you actually bought it. There should be a way to just see what you bought without the extra hassle of having to go to a different page. That is why we later updated it so that it also shows in a given company stock, specifically how many of it you bought. This way, there are two places where the user can see their stocks, and makes the user experience more comfortable than what was described before with the frequent back-and-forths. In terms of UI correctness, the current stock was never incorrect on either page, which simplified our efforts.

Deployment.

Deployed Link: https://virtrade-hpue.onrender.com/